

Machine Learning

Table of contents

1. What is Machine Learning?

- 1.1 Machine Learning Lifecycle
- 1.2 When to use Machine Learning
- 1.3 Types of Machine Learning
- 1.4 Regression
- 1.5 Classification
- 1.6 Clustering
- 1.7 Machine Learning Pipeline

2. Feature Engineering

- 2.1 Why is it important?
- 2.2 Types of Feature Engineering
- 2.3 Feature Transformation
 - 2.3.1 Feature Encoding
 - 2.3.2 Feature Scaling
- 2.4 Feature Extraction
- 2.5 Feature Selection

3. Regularization

- 3.1 Types of Regularization
 - 3.1.1 L1 Regularization
 - 3.1.2 L2 Regularization
 - 3.1.3 Elastic Net Regularization
- 3.2 Choosing Lambda

4. Supervised Machine Learning Models

- 4.1 Linear Regression
- 4.2 Polynomial Regression
- 4.3 Logistic Regression
- 4.4 Decision Trees

- 4.5 Support Vector Machines
- 4.6 K Nearest Neighbors

5. Errors in ML

- 5.1 Types of Error
- 5.2 Bias-Variance Tradeoff
- 5.3 What affects Bias and Variance
- 5.4 How to handle it?

6. Ensemble Methods

- 6.1 Why use Ensemble methods?
- 6.2 Types of Ensemble methods
- 6.3 Random Forest
- 6.4 Boosting Algorithms Comparison

7. Unsupervised Machine Learning Models

- 7.1 K Means Clustering
- 7.2 DBSCAN Clustering
- 7.3 HDBSCAN Clustering
- 7.4 Agglomerative Clustering

8. Principal Component Analysis

- 8.1 Dimensionality Reduction
- 8.2 PCA
- 8.3 Comparing PCA, t-SNE, UMAP

9. Hyperparameter Tuning

- 9.1 Model Evaluation
- 9.2 Data Splitting Strategy
- 9.3 Cross Validation
- 9.4 Methods of Hyperparameter Tuning

10. Model Evaluation

- 10.1 Classification Metrics
- 10.2 Regression Metrics
- 10.3 Clustering Metrics

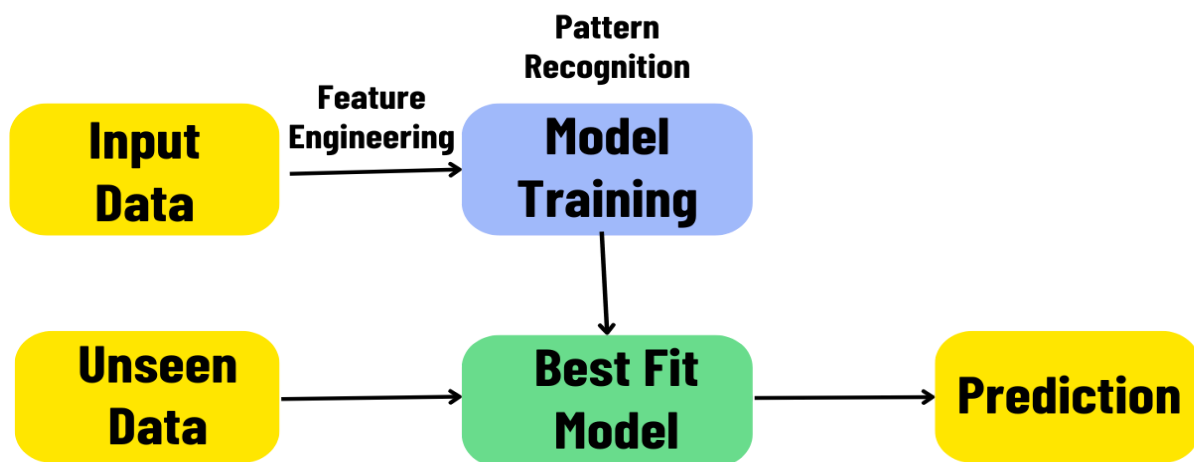
1. What is Machine Learning?

Machine Learning (ML), a branch of AI, is a way to teach computers to learn from data and make decisions or predictions without being explicitly programmed.

Think of it as:

Instead of writing rules for every situation, we let the computer find patterns from data and create its own rules.

1.1 Machine Learning Lifecycle



1.2 When to Use Machine Learning

- Rules are **too complex** or **unknown** to program manually
- There's a **lot of data** and patterns that can be learned
- You need the system to **improve over time**

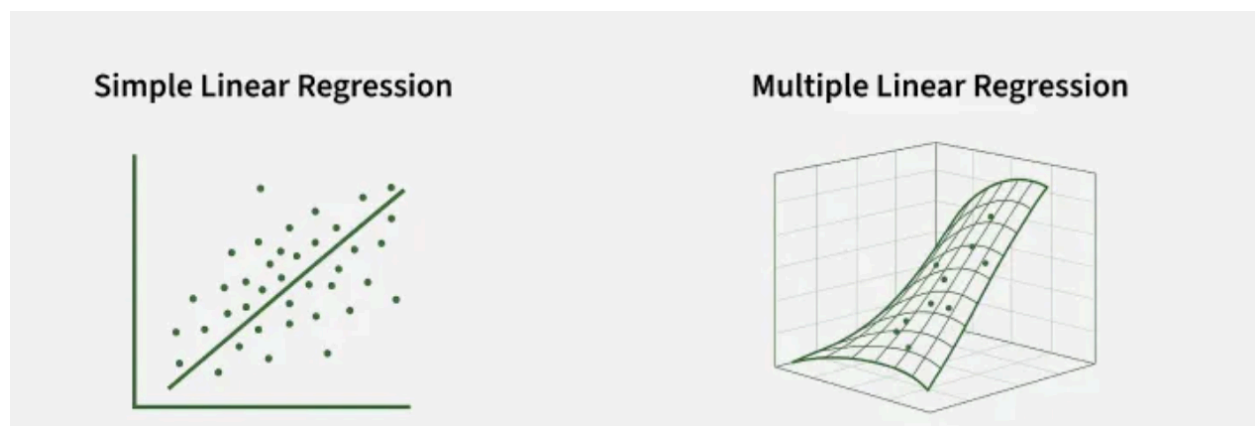
1.3 Types of Machine Learning

Type	Description	Real-Life Example
Supervised	Learn from labeled data	Spam email classification/ Disease Diagnosis
Unsupervised	Find hidden patterns in unlabeled data	Customer segmentation/ Anomaly detection
Reinforcement	Learn by trial-and-error	Game playing (like chess)/ self-driving cars

1.4 Regression

Regression is a type of supervised machine learning where the goal is to **predict a number (continuous value)** based on input features.

1.4.1 Types of Regression Problems



1. Simple Regression

- One input feature → one continuous output (straight line)
- Example: Years of Experience → Salary

2. Multiple Regression

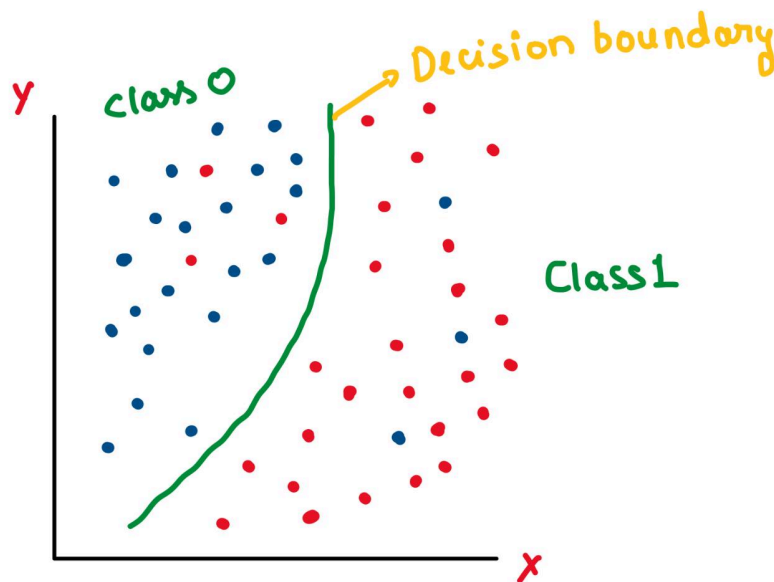
- Multiple input features → one continuous output (n dimensional plane where n is no. of features)
- Example: Experience + Skills + Education → Salary

1.4.2 Applications

- House Price Prediction
- Forecasting sales for next month
- Estimating salary based on experience
- Forecasting electricity consumption

1.5. Classification

Classification is supervised machine learning where the goal is to assign a label (class) to a given input based on its features.



1.5.1 Types of Classification

1. Binary Classification

- Predicts only two classes

- Example: Email → Spam(1) or Not Spam(0)

2. Multiple Classification

- Predicts more than two classes
- Example: Fruit → Apple, Banana, Orange

1.5.2 Applications

- Email filter as spam or not spam
- Medical diagnosis as disease or no disease
- Loan approval as rejected or approved
- Customer churn prediction as will leave or stay

1.6 Clustering

Clustering is an **unsupervised learning** technique used to group **similar data points** together based on patterns in the features, without using labeled outputs.

1.6.1 Types of Clustering

1. Partition Based Clustering

- Divides data into non overlapping clusters each represented by a centroid
- Identifies k clusters with minimal variance assigning each point to nearest centroid
- Efficient and scale well with large datasets
- Fails on identifying non-spherical clusters
- Sensitive to outliers

2. Density Based Clustering

- Groups data based on areas of high density of points (closely packed)
- Suitable for irregular shaped clusters
- Separates noise/ outliers from clusters
- No need to specify number of clusters

3. Hierarchical Clustering

- Builds a tree of clusters (dendrogram) by merging or splitting
- Contains smaller sub clusters
- Reveals relationship between clusters
- Computationally expensive
- Types:
 - **Agglomerative (Bottom-up):** Starts with each observation individual clusters, merges similar customer into larger parent
 - **Divisive (Top-down):** All observation in single root clusters, iteratively splits into smaller child clusters

1.6.2 Applications

- Customer Segmentation
- Social network analysis
- Anomaly detection
- Fraud detection
- Analyzing market segments

1.7 Machine Learning Pipeline

- 1. Problem Definition:** Clearly understand what you're solving
- 2. Data Collection:** Gather the right data from the right sources
- 3. Data Cleaning & Preprocessing:** Prepare the data so it can be used in ML models
- 4. Feature Engineering:** Create or select the most meaningful features that influence the output
- 5. Train and Test Split:** Create training set to train the model and test set to evaluate the model performance after training
- 6. Model Building:** Choose and train an algorithm that learns patterns in data
- 7. Model Evaluation:** Check how well your model performs on unseen data (test data)

8. Model Tuning & Optimization: Fine tune model parameters to increase model performance

9. Model Deployment: Take the trained model and use it in the real world (as an API, Embedded in Apps, or in Cloud)

10. Monitoring & Maintenance: Track how the model performs over time for Data drift (new data is different from training data), Performance decay, Need for re-training

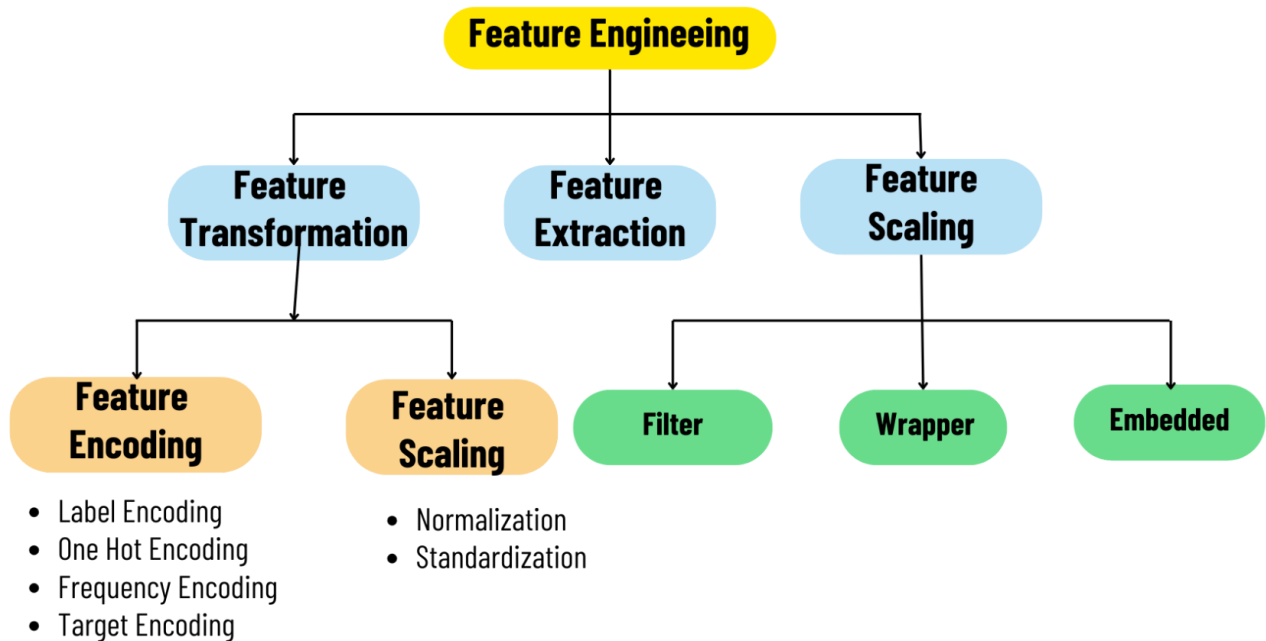
2. Feature Engineering

Feature Engineering is the process of creating, modifying, or selecting the most useful features from raw data to improve the performance of your ML model.

2.1 Why is it important?

1. An excessive number of features can lead to an **overfit model**, while too few features may result in an **underfit model**.
2. Certain models, such as K-Nearest Neighbors (KNN) and Support Vector Machines (SVMs), necessitate that predictor variables **share the same unit or scale** to prevent bias.
3. Some models exhibit high sensitivity to **correlations between predictors**.
4. Models do not understand **categorical data**. Understands only numbers
5. It facilitates the **development of new features** that represent something important to the outcome.

2.2 Types of Feature Engineering



2.3 Feature Transformation

Feature Transformation is the process of modifying existing features so they can be better understood and processed by ML algorithms.

Purpose:

- Convert raw data into a model-friendly format
- Improve model accuracy and learning speed
- Handle inconsistent, skewed, or categorical data
- Model Learns Faster and better

2.3.1 Feature Encoding

Convert non-numeric values (like City, Color, Gender) into numbers, because ML algorithms can only understand numbers, not strings or categories.

Types of Encoding

1. Label Encoding:

- Assigns each category a unique number
`{"Red" → 0, "Blue" → 1, "Green" → 2}`
- Use when categories are **ordinal** (have a meaningful order)
`{High School → 0, Bachelor → 1, Master → 2, PhD → 3}`
- Don't use this for nominal (unordered) categories — models may think higher numbers = better.

2. One Hot Encoding:

- Creates a new column for each category. Values: 1 (present) or 0 (absent)
- Best for **nominal** categories (no order)
- Can increase dataset size quickly with too many categories (dimensionality issue)
- Example:
`City = {Delhi, Mumbai, Kolkata}`
Creates: `City_Delhi, City_Mumbai, City_Kolkata`

Original Data (Before Encoding):

Person ID	City
1	Delhi
2	Mumbai
3	Kolkata
4	Delhi
5	Kolkata
6	Delhi

Encoded Data (After Encoding):

ID	City	City_Delhi	City_Mumbai	City_Kolkata
1	Delhi	1	0	0
2	Mumbai	0	1	0
3	Kolkata	0	0	1
4	Delhi	1	0	0
5	Kolkata	0	0	1
6	Delhi	1	0	0

3. Frequency Encoding:

- Replace each category with its frequency/count in the dataset
- Example: **Delhi = 3, Mumbai = 1, Kolkata = 2**
- Effective for high cardinality features but problem when two categories having same frequency

4. Target Encoding:

- Replace each category with the **mean value of the target variable** for that category (only for classification)
- Not suitable if there is any unseen category in new data/ test data
- Good for high cardinality features
- Risk of **data leakage** — apply on training data only with cross-validation

City	Mean Purchased (Target Encoding)
Delhi	$(1 + 0) / 2 = 0.5$
Mumbai	$0 / 1 = 0.0$
Kolkata	$(1 + 1) / 2 = 1.0$

City	Purchased	City_Encoded
Delhi	1	0.5
Mumbai	0	0
Kolkata	1	1
Delhi	0	0.5
Kolkata	1	1

2.3.2 Feature Scaling

Feature scaling brings all features to a **similar scale** to improve model performance because features with larger scales are given high priority (assigns more weight) by model leading to biasness.

- Use scaling when your model relies on distance based calculations. Example KNN, Linear Regression, SVM
- Not needed for tree based models (Decision Tree, Regression Forest, XGBoost) because they make decisions based on splitting data at specific thresholds, not on the absolute value of features

Types of Feature Scaling:

1. Normalization (Min-Max Scaling):

- Rescales all values in a fixed range between 0 and 1.
- Not good in case of outliers.

For feature X,

$$X_{i_norm} = \frac{X_i - X_{min}}{X_{max} - X_{min}}$$

Example:

Age = 25, min = 20, max = 60

→ Normalized = (25 - 20)/(60 - 20) = 0.125

2. Standardization (StandardScaler):

- Rescales data to have Mean = 0 and Standard deviation = 1
- Best for models assuming normal distribution (Logistic Regression, SVM)
- Handles outliers better than normalization

For feature X, where μ is mean and σ is standard deviation of X

$$X_{i_standardized} = \frac{X_i - \mu}{\sigma}$$

Example:

If height = 170, mean = 160, std = 10

→ Standardized = $(170 - 160) / 10 = 1.0$

2.4 Feature Extraction

Feature Extraction is the process of creating **new features** from raw data that highlight hidden patterns or compress important information.

- It's different from feature *selection* (which chooses from existing features).
- In **extraction**, we **create new ones** that are often more useful.

Why Use Feature Extraction?

- When raw data is **too detailed, noisy, or unstructured**
- To **reduce dimensionality** while retaining core information
- To convert **text, image, audio, or time-based data** into numbers

When to Use It?

- You're working with **text, time-series, or unstructured** data
- You want to combine or summarize multiple features

Types of Feature Extraction:

1. Manual Feature Extraction (Using Domain Knowledge)

Create new features using logic, experience, or real-world understanding.

Raw Feature	Extracted Feature
Date of Birth	Age

Timestamp	Day of week, Hour of day
Full Address	City, State, Zip Code
Review	Length of review, sentiment score

2. Binning (Discretization)

Group continuous numeric values into categories or buckets. It reduces noise and makes patterns easier to detect.

Type	Description	Example
Equal Width Binning	Divides values into equal ranges	Age: 0-20, 21-40, 41-60
Equal Frequency Binning	Equal number of data points in each bin	Top 25% sales -> High Next 25% -> Medium etc.
Custom Binning	Based on domain knowledge	Income: Low (<30k), Medium, High

3. Time-based Feature Extraction

From timestamps like 2024-06-07 14:30:00, you can extract:

- Hour of day
- Day of week
- Month
- Year
- Weekend/Weekday
- Time since last event
- Quarter

2.5 Feature Selection

Feature Selection is the process of identifying and selecting only the **most relevant features** from your dataset to train a machine learning model.

Purpose:

- Improve model performance
- Make the model simpler and easier to interpret
- Speed up training
- Reduce overfitting

When to Use Feature Selection?

- Your dataset has too many features
- You suspect some features are correlated or irrelevant
- You're facing overfitting or low accuracy

Types of Feature Selection

1. Filter Methods

- Independent of any model. Use statistics to select features.
- Fast and Simple

Common Techniques:

- **Correlation Matrix** (for numeric features)
 - Drop features highly correlated with each other and less correlated with target variables.
- **Chi-Squared Test** (for categorical + target)
 - Select features most associated with the output
- **ANOVA F-Test** (for numeric + categorical target)
 - Measures variance between groups

2. Wrapper Methods

- Use a model to test different combinations of features.

- Computationally expensive for large datasets
- Works well with models like Logistic Regression, Decision Trees, etc.

Common Techniques:

- **Forward Selection**
→ Start with no features, add one at a time based on performance
- **Backward Elimination**
→ Start with all, remove one at a time
- **Recursive Feature Elimination (RFE)**
→ Repeatedly remove least important features based on a model

3. Embedded Methods

- Feature selection happens inside the model training.
- Model-specific, more efficient than wrapper method.

Common Techniques:

- **Lasso Regression (L1 Regularization)**
→ Shrinks irrelevant feature weights to zero
- **Tree-based feature importance**
→ Decision Trees, Random Forests, and XGBoost rank feature importance

3. Regularization

Regularization is a technique used to **prevent overfitting** in machine learning models by **penalizing large coefficients** by reducing their magnitude.

- It adds a penalty term to the loss function
- This discourages the model from assigning large weights to features

- The more complex the model (larger weights), the bigger the penalty.

Regularized Loss = Loss function + $\lambda \times$ Penalty,

Where λ is Regularization parameter (controls penalty strength)

3.1 Types of Regularization

3.1.1 L1 Regularization (Lasso)

- Adds the absolute value of coefficients as penalty
- Some weights become exactly zero, removing irrelevant features affect, helps in feature selection
- Useful when only few features are important
- Good for sparse models

3.1.2 L2 Regularization (Ridge)

- Adds the square of coefficients as penalty
- Shrinks all weights towards zero, but not exactly zero
- Useful when all features are useful
- Reduce model variance

3.1.3 Elastic Net Regularization

- Combines L1 and L2 penalties
- Balances feature selection and coefficient shrinkage
- Removes irrelevant features and reduce Variance
- Works well when features are correlated

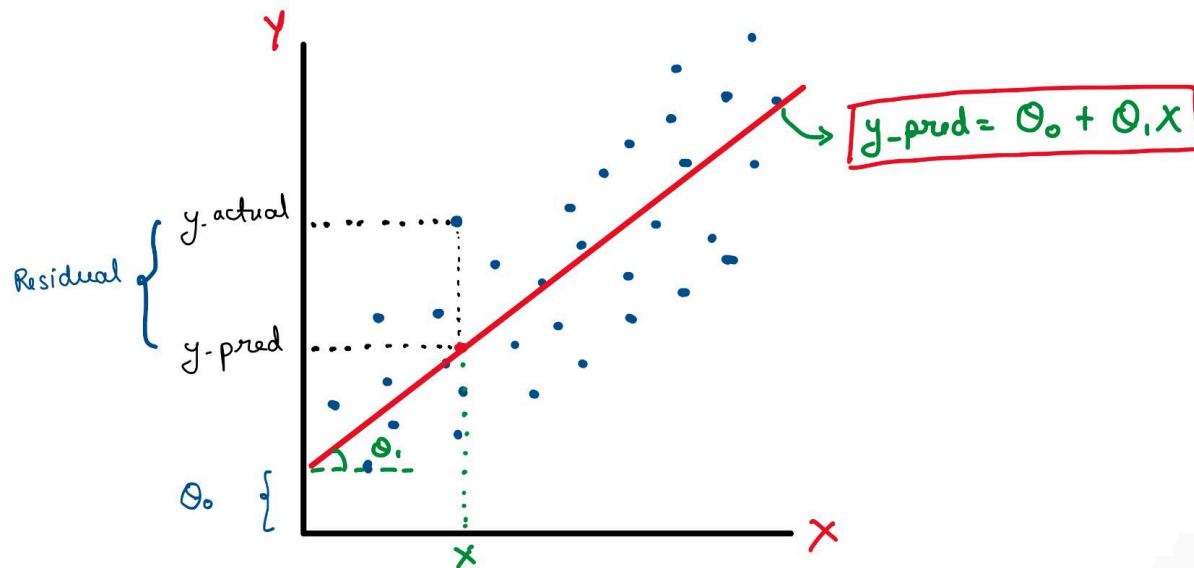
3.2 Choosing Lambda (λ)

- Small λ -> Weak Regularization (Little penalty) -> Overfitting risk
- Large λ -> Strong Regularization (Too much penalty) -> Underfitting
- Use cross validation to choose optimal λ .

4. Supervised Machine Learning Models

4.1 Linear Regression

Linear Regression is used to predict a **continuous numerical value** based on one or more input features using a straight line. It's used when the relationship between input and output is **roughly linear**.



- Linear regression draws the **best fit line** using Least Squares technique.
 - This method aims to minimize the sum of the squared differences between the actual data points and the values predicted by the line.
 - The model then uses this line to **predict future values**.
 - Loss Function: The loss function tells the model how wrong its predictions are. Linear Regression uses **MSE** to minimize errors.
-
- **Simple Linear Regression:** $y_{pred} = \theta_0 + \theta_1 X$
 - **Multiple Linear Regression:**

$y_{\text{pred}} = \theta_0 + \theta_1 X_1 + \theta_2 X_2 + \theta_3 X_3 \dots \theta_n X_n$, for n no. of features

- **Residual** = $y_{\text{actual}} - y_{\text{pred}}$
- **Least Squares** = $\Sigma(y_{\text{actual}} - y_{\text{pred}})^2$

Assumptions of Linear Regression

- Linear Relationship between inputs and output
- No multicollinearity (input features shouldn't be too correlated)

Training Steps

- Start with random values for slope and intercept (θ_0, θ_1)
- Predict values using $Y = \theta_0 + \theta_1 X$
- Compare predictions with actual using MSE
- Use an optimizer (like Gradient Descent) to reduce error
- Repeat until error stops improving

Advantages:

- Easy to understand and implement.
- Fast to train even on large datasets
- Doesn't require any tuning

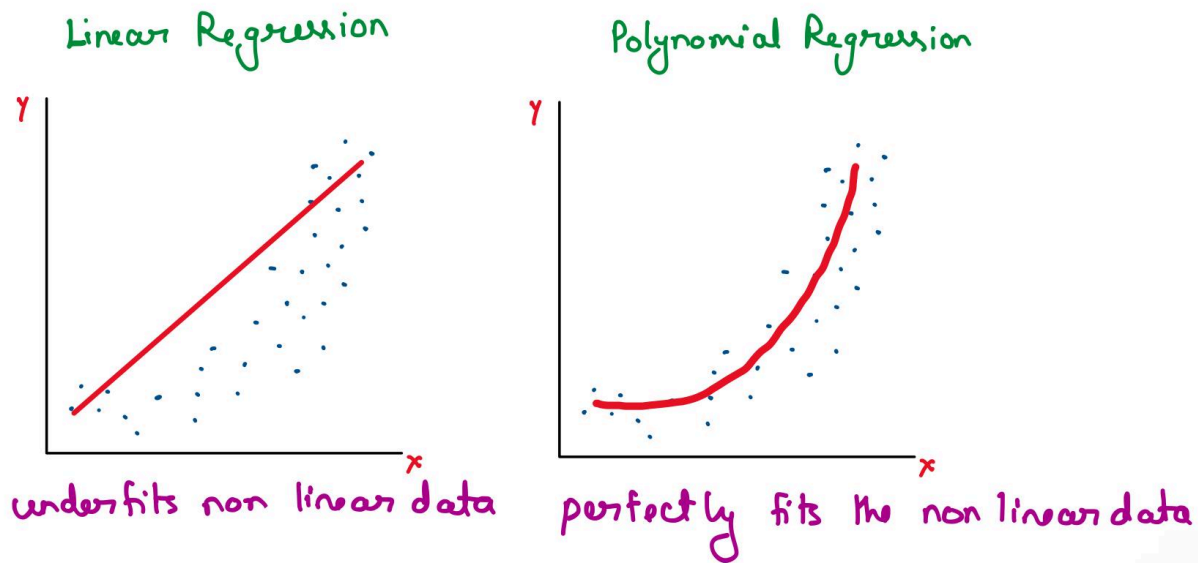
Disadvantages:

- Assumes a linear relationship (not always true in real time)
- Very sensitive to outliers
- Doesn't perform well if input features are highly correlated
- Can underfit complex patterns in data (always tries to fit straight line)

4.2 Polynomial Regression

Polynomial Regression is used to model the relationship between the independent variable(s) and the dependent variable when the relationship is **non-linear**.

Think of it as linear regression with extra powers of X (like X^2 , X^3) to better fit the curved pattern data.



$$Y = b_0 + b_1 \cdot X + b_2 \cdot X^2 + b_3 \cdot X^3 + \dots + b_n \cdot X^n$$

These added powers of X let the model capture curves, bends and non linear patterns.

X^2 , X^3 , ... = new features representing higher-order terms (polynomial features)

The model used in Polynomial Regression is a linear regression model but the features changed to polynomial features.

Training

- Add new features like X^2 , X^3 , etc. to your dataset
- Use regular **linear regression** to fit those features
- Find the best curve that **minimizes error** (like MSE)

Advantages:

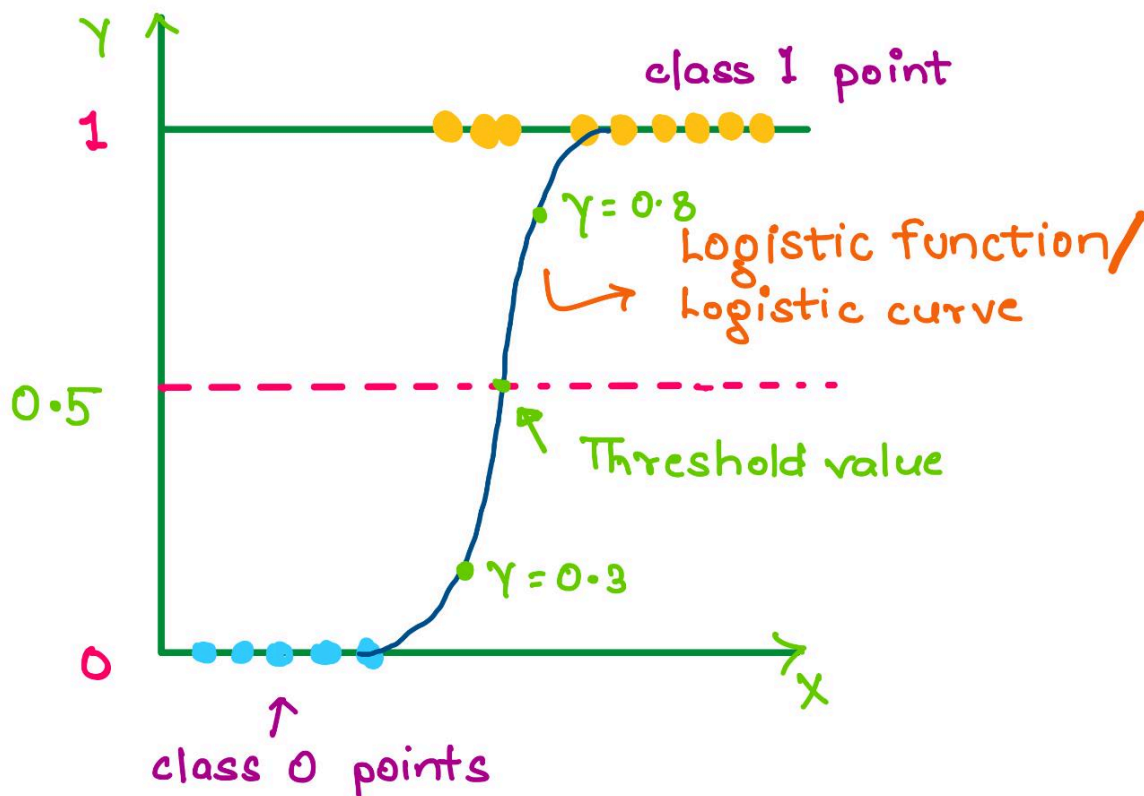
- Can model non-linear relationships easily
- Still uses linear model internally (simple to implement)

Disadvantages:

- Can create weird curves outside the data range
- Sensitive to outliers
- High degrees = more overfitting risk

4.3 Logistic Regression

Logistic Regression is a classification algorithm used to classify data into two categories.



- Instead of predicting a number (like linear regression), logistic regression predicts a **probability**
- That probability is converted into a **class (0 or 1)** using a **threshold** (commonly 0.5)

Example: If model predicts 0.8 → classified as **1 (Yes)**

If model predicts 0.3 → classified as **0 (No)**

- It calculates a score similar to Linear Regression: $y_{\text{pred}} = \theta_0 + \theta_1 X$

- This score is passed through a sigmoid function, which squashes it to a value between 0 and 1

$$\text{Probability} = \frac{1}{(1 + e^{-y_{pred}})}$$

- Loss Function: Binary cross entropy or log loss. Used to measure how far off the prediction is from the actual value.

Training

- Start with random weights for slope and intercept (θ_0, θ_1)
- For each data point:
 - Compute $z = mX + b$
 - Use sigmoid to get probability
 - Compare with actual value (0 or 1)
- Use cross-entropy loss to measure error
- Update weights using gradient descent to reduce error

Advantages:

- Works well on linearly separable data
- Fast to train and predict, easy to understand and implement

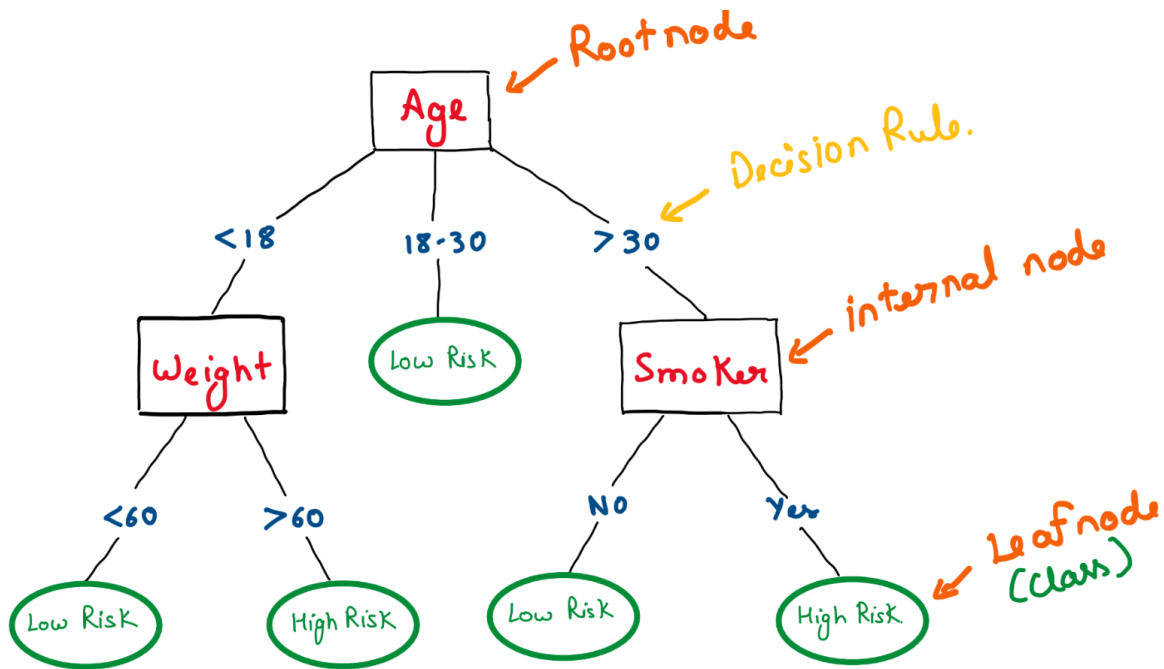
Disadvantages:

- Struggles with non-linear relationships
- Sensitive to outliers
- Not ideal when features are highly correlated
- Doesn't handle complex decision boundaries well

4.4 Decision Trees

A **Decision Tree** is a **supervised learning algorithm** used for both **classification** and **regression** tasks. It mimics human decision-making by breaking down a dataset into smaller subsets while building a tree structure.

- If you're classifying → **Classification Tree**
- If you're predicting a number → **Regression Tree**



- Root Node represents the best feature to split.
- Each **internal** node represents a **feature**.
- Each **branch** represents a **decision rule**
- Each **leaf node** represents an **outcome** or prediction.
 - For classification: leaf gives class label
 - For regression: leaf gives numerical value (average of samples)

Training

1. Start with a root node.
2. Train the node on its assigned data by finding the feature that splits the data into its pre-labeled classes.
3. Each split partitions the node's input data and each partition passes along its branch to a new node.
4. Repeat the process for each new node using each feature only once until it runs out of features to select or a pre-selected stopping criterion is met (pruning).

Tree Pruning

A deep decision tree may overfit the training data, capturing noise and anomalies.

Pruning reduces the size of the tree by removing less informative branches, making it more generalizable and accurate. It will stop growing trees when a stopping criterion is met.

- Maximum depth is reached
- Minimum number of data points in a node has been exceeded
- Minimum number of samples in a leaf has been exceeded
- Maximum number of leaf nodes reached

What is the Best Feature?

The best feature is the one that splits the data in a way that results in the purest possible child nodes.

We measure this purity using:

- For classification (how well a feature separates classes):
 - Entropy
 - Information Gain
- For regression (how close predictions are to actual values):
 - MSE
 - MAE

Entropy

Measure of information disorder in a dataset.

- If entropy = 0 → node is pure (only one class)
- Higher entropy → more disorder (mixed classes)

Information Gain

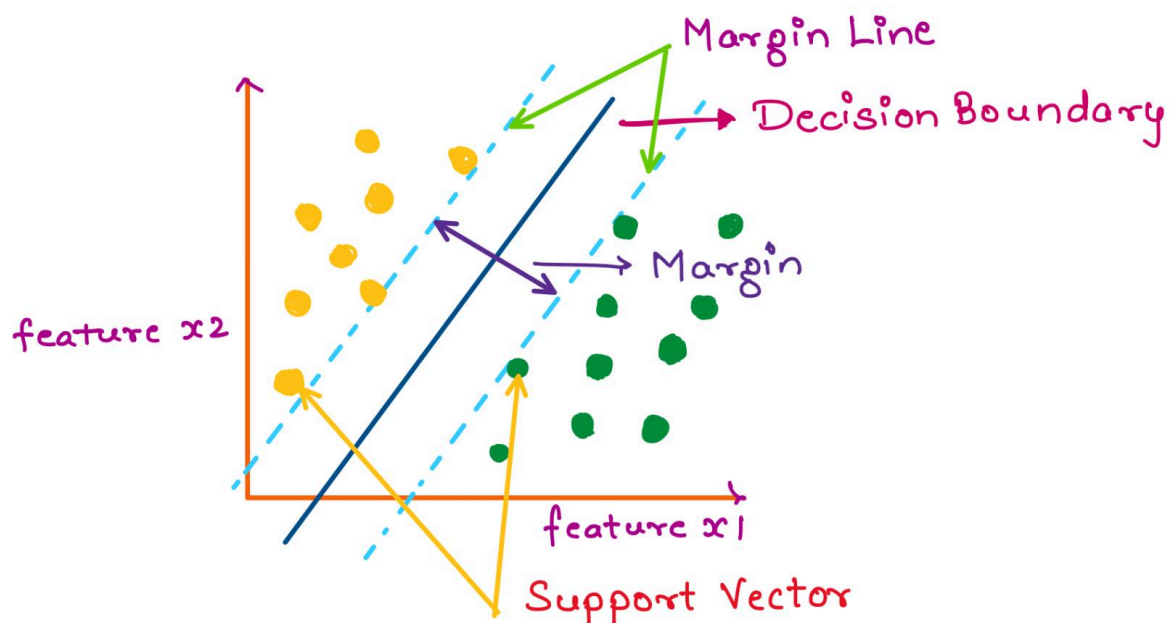
Measures how much entropy is reduced by splitting the data on a particular feature.

- Higher IG $\rightarrow$ better feature for splitting

4.5 Support Vector Machines

A **Support Vector Machine (SVM)** is used for both classification (**SVC**) and Regression (**SVR**)

- SVM tries to find a **hyperplane** that divides the data into two classes with the widest possible gap (**margin**) between the closest points (support vectors) of each class.
- SVR tries to fit a function within a margin of tolerance (**epsilon**) from the true outputs.



- The larger the margin, the better the accuracy. The above figure shows linear separable classes.
- But in real time, data is noisy, overlapping, non linear making perfect separation difficult.
- For this, it uses SVM uses **Soft Margin** and **Kernels**

Soft Margins

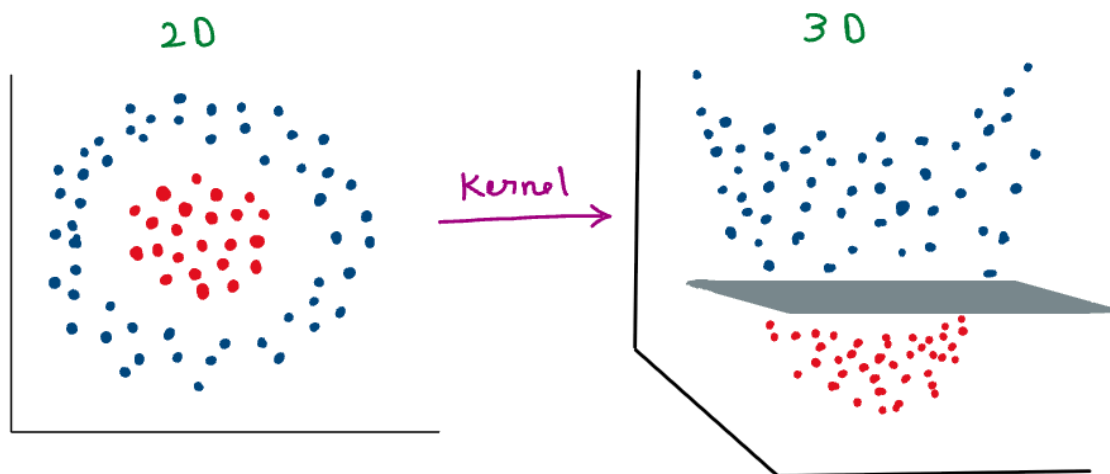
It allows some misclassifications while maximizing the margin

Controlled by **C** parameter (regularization)

- **Low C** → soft margin, more misclassifications (high bias)
- **High C** → hard margin, fewer errors (risk of overfitting)

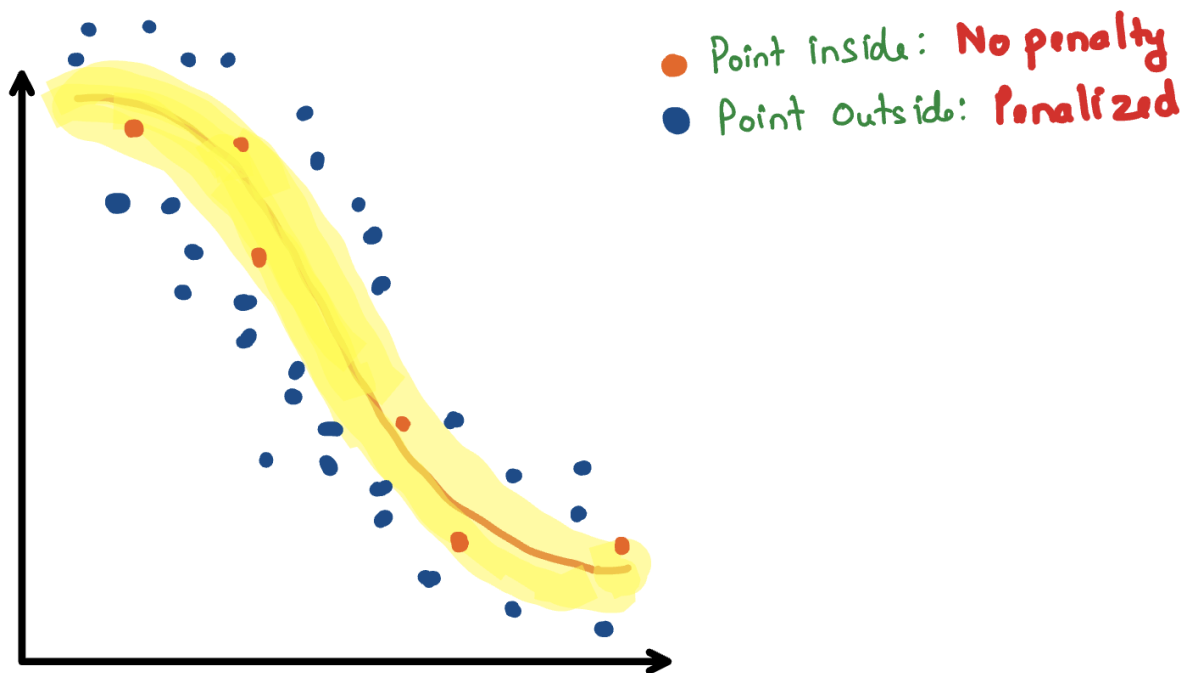
Kernels

Real-world data is rarely linearly separable. So, SVM uses **kernel tricks** to transform data into higher dimensions where a hyperplane can separate them.



- **Linear**: when data is linearly separable
- **Polynomial**: when data has polynomial relations
- **RBF (Radial Basis Functions)**: most commonly used for complex data
- **Sigmoid**: less common, similar to neural network activation functions

Epsilon



Epsilon is a parameter of SVR learning algorithm that you can select to define a margin around the prediction curve.

Epsilon is the maximum allowable error between predicted and actual where you don't penalize the model.

Point inside: No penalty

Point outside: Penalized (adds to loss function)

Smaller Epsilon -> Overfit -> Tighter tube

Larger Epsilon -> Allow some error -> Smoother model (less sensitive to noise)

Advantages:

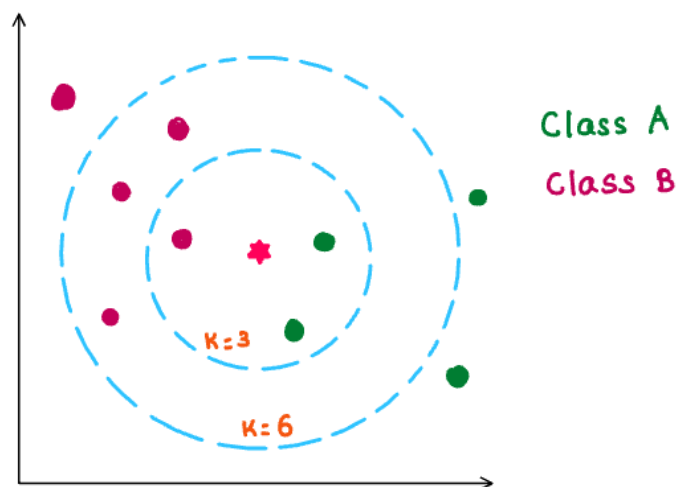
- High accuracy in high dimensional spaces
- Can handle linear as well as non linear data

Disadvantages:

- Poor performance on large datasets
- Requires careful tuning of hyperparameters
- Doesn't scale well with noisy or overlapping classes
- Harder to interpret than decision trees

4.6 K Nearest Neighbors

KNN is used for both classification and regression problems that predicts a value or class for a new data point based on how its nearest neighbors are labeled (majority or average of nearest data points).



It's also called a **lazy learner** because:

- No computation during training (No training)
- It memorizes training data
- All computation happens at prediction time (when a new input is given) based on its distances to all points in training data

Working

1. Pick a value for K (number of neighbors)
2. For each query point
 - a. Measure the distance between the input point and all training samples
 - b. Select K nearest neighbors (top k smallest distances)
 - c. For Classification : Pick the majority class among neighbors

- d. For Regression: Take the average/ median (in case of outliers) of neighbors target values
- 3. In case of **skewed** and **imbalance class**, new predictions will belong to the majority class creating biasness. Majority class will dominate prediction

Solution:

- **Weight Classification:** Assign weights to neighbors based on distance or class frequency. Closer neighbors gets more influence
- **Resampling:** Oversampling or undersampling

Advantages:

- No training time, faster
- Works well with small datasets
- Adapts well to new data (just re run search)

Disadvantages:

- Requires lot of memory (stores entire dataset)
- Doesn't work well with imbalanced data
- Slow prediction on large datasets (computes distance from all points)
- Sensitive to irrelevant/noisy features
- Requires feature scaling
- Suffers from curse of dimensionality (many features = poor performance) as dimensions increase, distance become less meaningful

5. Errors in ML

Errors in machine learning tell how accurately our model can predict new data.

5.1 Types of Error:

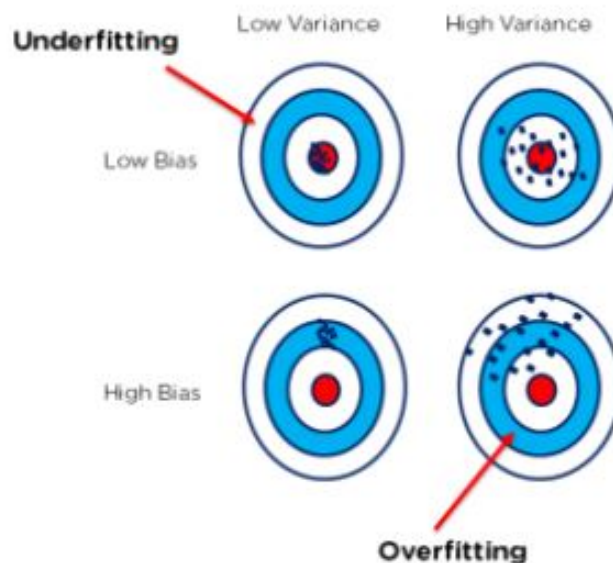
1. Irreducible Error: This is the **noise** in the data that **cannot be removed**, no matter how good your model is. All models will have some irreducible error.

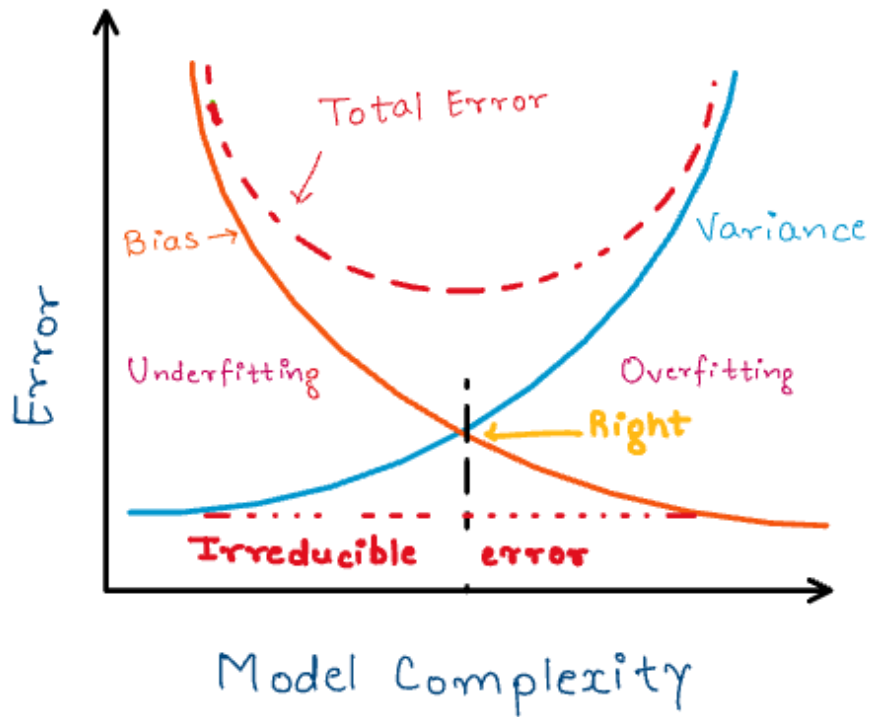
2. Reducible Error: This is the part of error that **can be reduced** by choosing better models, features, or techniques.

It has two components:

- **Bias:** Differences between actual and predicted values.
High bias = model is too simple to capture the complexity of data (underfitting)
- **Variance:** Model's sensitivity to small fluctuations when trained on small subsets of same training data
High Variance = model is too complex, captures noise as if it were a pattern (overfitting)

5.2 Bias-Variance Tradeoff





Model Complexity	Bias	Variance	Total Error
Simple Model	▲ High	▼ Low	High (underfit)
Complex Model	▼ Low	▲ High	High (overfit)
Generalized Model	✓ Balanced	✓ Balanced	✓ Low

Tradeoff is about finding the sweet spot between underfitting and overfitting to achieve good generalization on unseen data.

5.3 What affects Bias and Variance?

- **Model Complexity:** More complex, Low Bias, High Variance
- **Training data size:** More data, Low Variance
- **Regularization:** Controls variance by simplifying model
- **Feature engineering:** Better features, Low Bias, Low Variance
- **Noise in data:** High Variance

- **Cross-Validation:** Helps detect High Variance

4.4 How to handle it?

- Cross Validation
- Feature Engineering
- Ensemble Methods

6. Ensemble Methods

Ensemble learning is a technique where **multiple models** (called “**weak learners**”) are combined to create a **stronger model**.

Instead of using **one predictor**, we use **many**, and then **aggregate** their outputs for better performance.

6.1 Why use Ensemble methods?

- A single model may be unstable, inaccurate, or biased.
- By combining multiple models, we can:
 - Improve accuracy
 - Reduce overfitting and underfitting
 - Handle complex problems

6.2 Types of Ensemble methods

1. Bagging (Bootstrap Aggregation)

Train **multiple models** on **different random subsets** of the data (with replacement) in parallel and **average their predictions** (for regression) or **vote** (for classification).

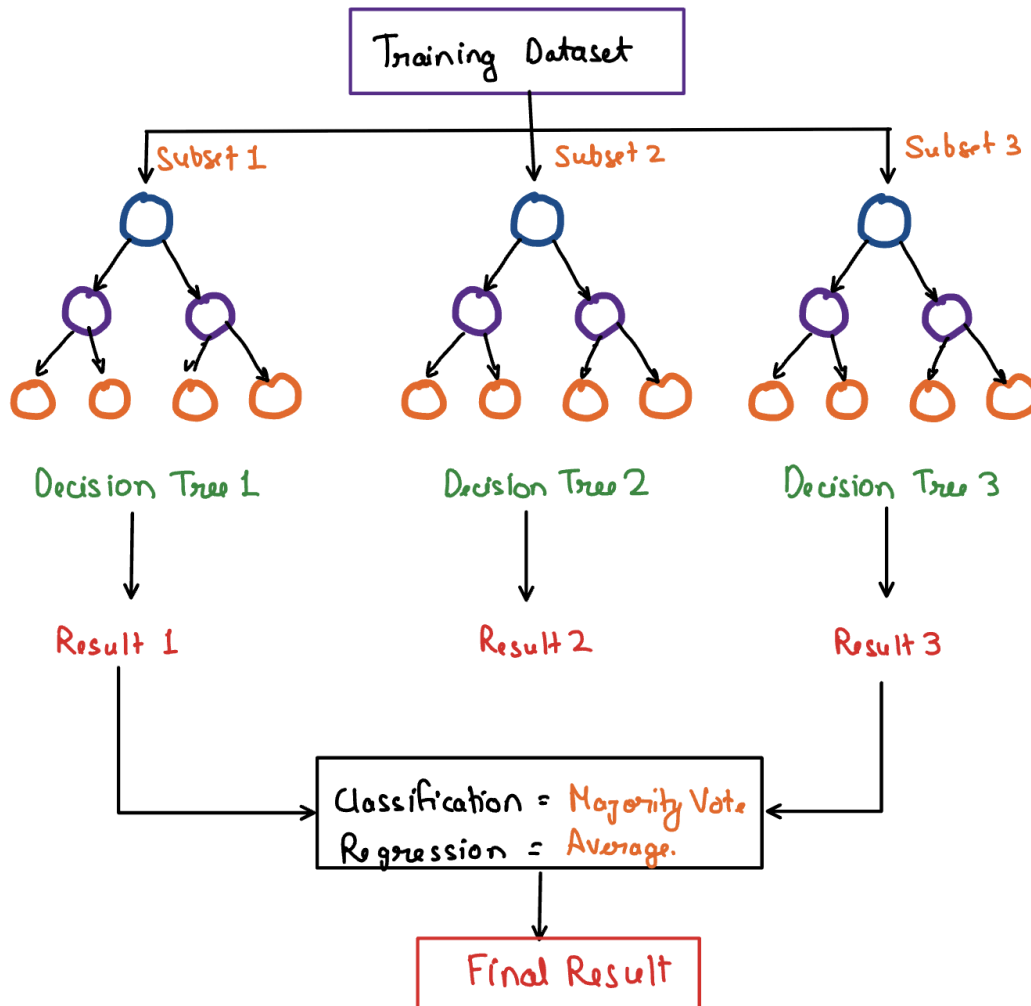
Algorithms: Random Forest

Objective: Reduce Variance and prevents overfitting

Working

- Create several **bootstrapped datasets** (random samples with replacement).

- Train **independent models** on each subset.
- Combine results (majority vote or average).



2. Boosting

Train models **sequentially**, each one **focusing more on the mistakes** made by the previous ones.

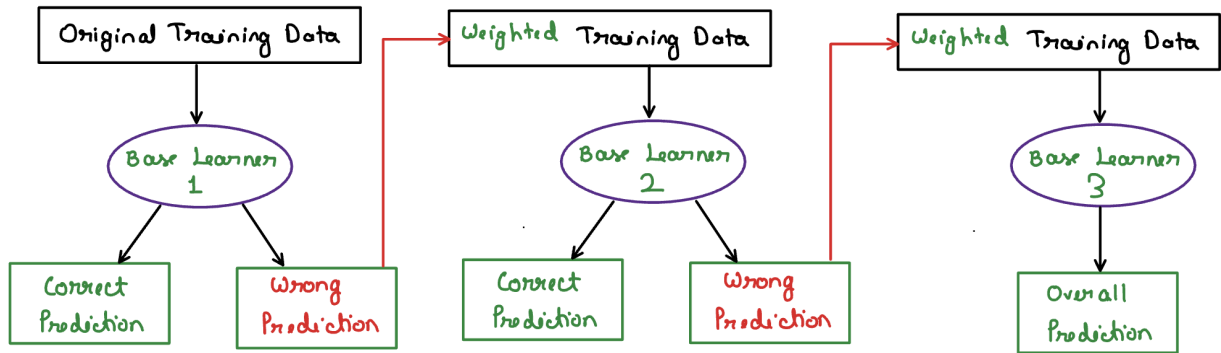
Algorithms: AdaBoost, GradientBoost, XGBoost

Objective: Reduce both Bias and Variance, prevents underfitting

Working

- Train the first model.
- Identify errors/ misclassification points

- Give more weight to those difficult points and decrease weights for correctly classified data.
- Train the next model to focus on correcting mistakes
- Model weights updates on each weak learners performance
- Final prediction is weighted combination of all models



6.3 Random Forest

Random Forest is an ensemble learning algorithm that builds multiple decision trees and combines their results to make a more robust and accurate prediction. It's a type of Bagging (Bootstrap Aggregation) method.

Working

- Each tree is trained on a random subset of the training dataset (with replacement).
- At each split in the tree, only a **random subset of features** is considered, not all.
- Each tree is slightly different because of randomness in data and features.
- **Classification**: Each tree gives a class → final prediction = **majority vote**
- **Regression**: Each tree gives a value → final prediction = **average**

Advantages:

- Reduces overfitting (compared to a single decision tree)
- Handles high-dimensional data well

- Works with missing or unscaled data
- Less sensitive to noisy data
- Automatic feature importance

Disadvantages:

- Can be memory-intensive if too many trees are built.
- Slower than a single tree
- Can overfit with too many trees (though less likely)

6.4 Boosting algorithms Comparison

Feature	AdaBoost	Gradient Boosting (GBM)	XGBoost (Extreme Gradient Boosting)
Core Idea	Focus on misclassified samples	Minimize loss function using gradient descent	Optimized GBM with regularization and speed improvements
Training Type	Sequential weak learners	Sequential learners (gradient descent)	Sequential learners + optimized tree boosting
Loss Function	Exponential loss	Customizable (MSE, Log Loss, etc.)	Customizable (MSE, Log Loss, etc.)
Handles Bias/Variance	Reduces Bias	Reduces Bias	Reduces Bias and Variance
Regularization	No	No	Yes (L1 & L2 regularization)
Weight Updates	Sample weights updated in each round	Uses gradients (errors) to fit next model	Same as GBM + second-order derivatives
Speed	Slower on large datasets	Medium	Faster (parallel training, hardware optimization)
Memory Usage	Low	Medium	Higher (due to extra computations and caches)
Hyperparameter Tuning	Fewer parameters	Moderate	Many (but powerful control)

Overfitting Handling	Prone to overfitting on noisy data	Prone if not tuned	Better handling with regularization and pruning
Performance	Good for clean data, fewer outliers	Better performance than AdaBoost	Best performance for large, complex datasets
Robustness to Noise	Low (sensitive to outliers)	Medium	High (can ignore outliers via regularization)

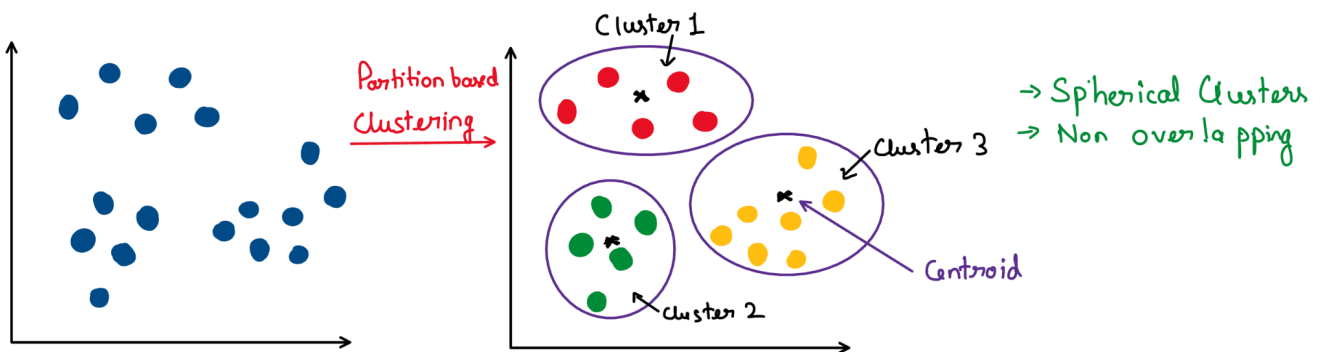
7. Unsupervised Machine Learning Models

7.1 K Means Clustering

K-Means is a **partition-based or centroid-based unsupervised learning algorithm** used to group data into **K distinct, non overlapping clusters** based on similarity.

Each group is formed by minimizing the **distance between data points and the center** of the group (called the **centroid**).

- Higher K = Smaller Clusters with greater detail
- Lower K = Larger Clusters with less detail



Training

1. Choose K (number of clusters to form).
2. Initialize K centroids randomly (starting cluster centers).
3. Assign points to the nearest centroid (form clusters).
4. Recalculate centroids: Find the center (mean) of each new cluster.
5. Repeat until:
 - a. Centroids stop moving much (stabilize)
 - b. Max iterations reached

Choose Optimal Value of K

- 1. Visualization:** Plot data points as scatter plot to see how many clusters can be formed
- 2. Elbow Method:**
 - a. Plot inertia (sum of squared distances to centroids) vs K (number of clusters)
 - b. Look for the “elbow” point, where the rate of decrease slows down.
 - c. The K at this point is considered optimal.
- 3. Silhouette Score:**
 - a. Measures how similar a point is to its own cluster vs. other clusters.
 - b. The score ranges from -1 to 1.
 - c. Higher silhouette score = better cluster separation.

Advantages:

- Simple and fast to implement.
- Works well when clusters are spherical and well-separated.
- Scales to large datasets.
- Efficient with low computational cost.

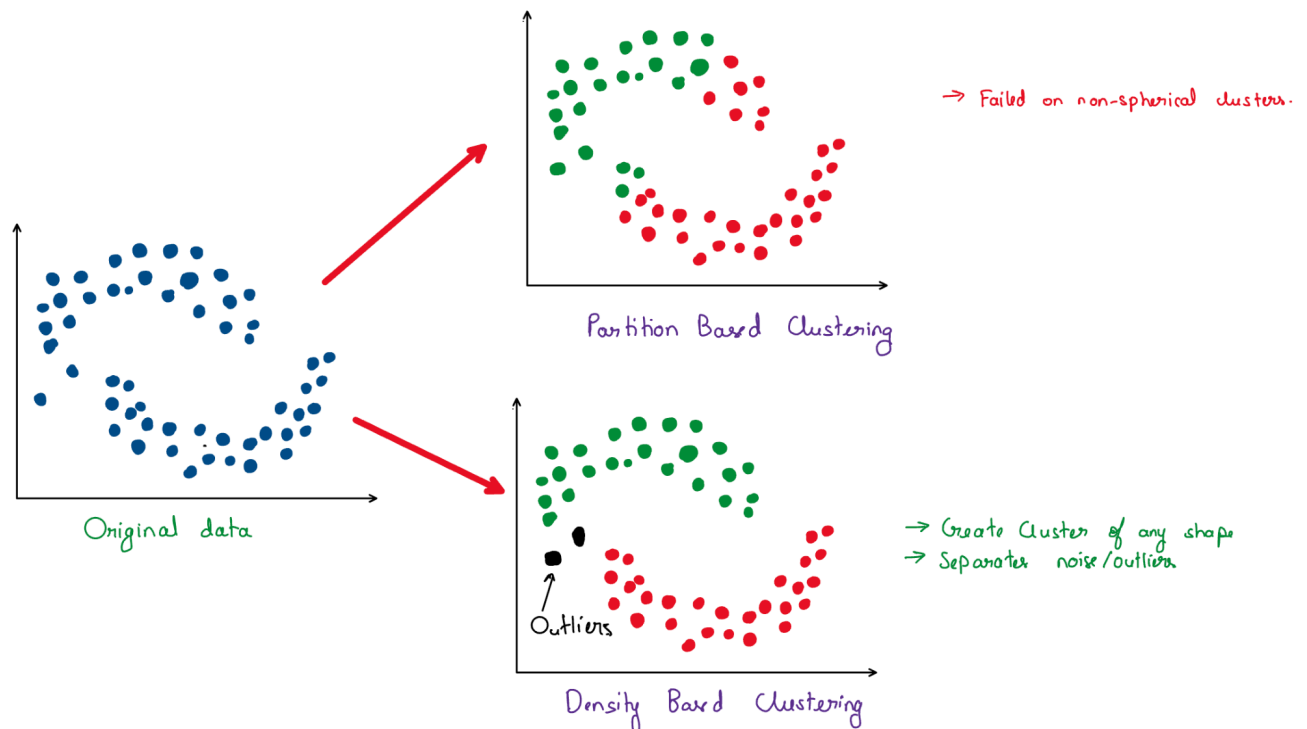
Disadvantages:

- You need to predefine K (number of clusters).
- Struggles with irregular-shaped clusters.
- Sensitive to outliers (can skew centroids).
- Initial centroid selection can affect results (may get stuck in local minima).

7.2 DBSCAN Clustering

DBSCAN is a **density-based clustering algorithm** that forms clusters based on the **density of data points**, and **ignores outliers**.

- It is especially useful when clusters are **not clearly separated**, **not spherical**, or contain **noise**.
- It also identifies **outliers (noise)** that do not belong to any cluster.
- It does **not** require you to predefine the number of clusters.



Training

1. Choose two parameters:
 - `eps`: the maximum distance to consider a neighbor
 - `min_samples`: the minimum number of points required to form a dense region
2. For each point in the dataset:
 - Count how many points fall within `eps` distance.
 - If $\text{count} \geq \text{min_samples}$, it's a core point.

- Group all core points and connected border points into a cluster.
 - If a point is not reachable from any cluster → label it as noise.
3. Output:
- Cluster labels for each point
 - Some points may be marked as -1 (noise)

Advantages:

- No need to specify K
- Can find clusters of any shape (non-spherical)
- Detects noise/outliers automatically

Disadvantages:

- Needs careful tuning of eps and min_samples
- Doesn't scale well to very large datasets
- Struggles when density varies across clusters
- Performance may drop in high-dimensional spaces

7.3 HDBSCAN Clustering

HDBSCAN stands for **Hierarchical Density-Based Spatial Clustering of Applications with Noise**.

- It is an improved, smarter version of DBSCAN, especially powerful when your data has clusters of varying densities.
- It does not need a fixed density threshold (epsilon), automatically adapts to different densities.

Training

1. Builds a hierarchy of clusters based on point density, instead of relying on a single density value.
2. Creates a minimum spanning tree of the data using distance and density.
3. Condenses this hierarchy to find the most stable clusters.
4. Removes noise points that don't belong to any consistent cluster.

Advantages:

- No need to specify K, can identify outliers
- Works with varying density clusters
- No need to define eps (unlike DBSCAN)
- Suitable for large and noisy real-world datasets

Disadvantages:

- Slightly slower than DBSCAN
- Harder to visualize hierarchy for large data

7.4 Agglomerative Clustering

Agglomerative Clustering is a **bottom-up hierarchical clustering** method.

It starts by treating **each data point as its own cluster**, then **merges the closest pairs** step-by-step until only one big cluster remains (or you get desired number of clusters)

Training

1. Start with N clusters (each data point is its own cluster).
2. Compute the distance (or similarity) between all clusters.
3. Merge the two closest clusters.
4. Repeat steps 2–3 until:
 - All points are in one cluster
 - Or a predefined number of clusters is reached

Advantages:

- No need to specify the number of clusters upfront (can be chosen by dendrogram)
- Works well with non-spherical clusters
- Can capture nested clusters and hierarchical relationships

Disadvantages:

- Not scalable for large datasets (time-consuming for thousands of points)
- Sensitive to noise and outliers

8. Principal Component Analysis (PCA)

8.1 Dimensionality Reduction

Problems with High Dimensional Data

- High dimensional data poses challenges for distance based algorithms (K Means, DBSCAN).
- As dimensionality increases, volume expands rapidly, causing data points to become sparse, leading to smaller clusters.
- Clustering results can not be visualized directly when working with more than 3 features/ dimensions

Dimensionality Reduction is the process of reducing the number of input features (dimensions) in your dataset. The goal is to retain as much important information as possible while making the data simpler, faster, and easier to analyze or visualize.

Techniques: PCA, t-SNE, UMAP

8.2 PCA

- PCA is a **linear dimensionality reduction** technique.
- Assumes dataset are linearly correlated.
- It transforms the data into **new axes (principal components)** that capture the most **variance** (important information) in the data.
- Principal Components are orthogonal to each other.
- The **first component** captures the most variation, the second captures the next most, and so on.

8.3 Comparing PCA, t-SNE, and UMAP

Feature	PCA	t-SNE	UMAP
Type	Linear	Non-linear	Non-linear
Speed	Fast	Slow	Faster than t-SNE

Preserves	Global structure	Local structure	Local + some global structure
Best For	Preprocessing, feature reduction	Visualization (2D, 3D)	Visualization + downstream tasks
Number of Dimensions	Any	Usually 2 or 3	Flexible (2D to many)
Scalability	Good for large datasets	Poor	Better than t-SNE
Use Case Example	Speeding up ML models	Visualizing word embeddings	Visualizing and clustering text

9. Hyperparameter Tuning

Hyperparameters are the **settings** of the model you choose manually **before training** (like number of trees in a random forest, learning rate, k in KNN, etc.)

Hyperparameter tuning is the process of finding the **best settings** to improve performance.

9.1 Model Validation

Model validation is the process of testing how well your machine learning model performs on unseen data. Trying to optimize the model without compromising it prevents overfitting when selecting the best configuration by tuning.

Validation set

It is the subset of a training set, used during model optimization to evaluate a model's performance by tuning model settings (hyperparameters)

9.2 Data Splitting Strategy

Set	Purpose
Training Set	Used to train the model
Validation Set	Used to fine-tune hyperparameters
Test Set	Used to evaluate final model

9.3 Cross Validation

Cross-validation is a smarter way to validate your model, by using all your data more effectively.

It splits the dataset into multiple folds and trains + validates multiple times.

9.3.1 Types of Cross Validation

1. K-Fold Cross Validation

- Split the data into K equal parts (folds)
- Use K-1 folds for training, and 1 fold for validation
- Repeat this K times, each time changing the validation fold
- Average the performance across all K runs
- Select the best model

2. Stratified K-Fold Cross Validation

- Like K-Fold but preserves class proportions in each fold, useful for classification
- Used in case of imbalance classification.

3. Time Series Split

- Used for time-series data where order matters. Ensures training data always comes before test data.

9.4 Methods of Hyperparameter Tuning

Grid Search: Try all combinations of hyperparameters

- Define a grid (dictionary) of hyperparameters and their possible values.
- Train and evaluate the model for all combinations.
- Select the combination with the best performance.

Random Search: Try random combinations (faster than grid search)

- Define the search space.
- Choose a fixed number of iterations (e.g., 10).
- Randomly select parameter combinations and evaluate.
- Pick the best-performing configuration.

10. Model Evaluation Metrics

10.1 Classification Metrics

Metric	What It Is / Use	How It's Calculated
Accuracy	Overall correctness of the model; best when classes are balanced.	$(TP + TN) / (TP + TN + FP + FN)$
Precision	Out of all predicted positives, how many were truly positive; useful in spam detection .	$TP / (TP + FP)$
Recall (Sensitivity)	Out of all actual positives, how many were correctly predicted; important in medical diagnosis .	$TP / (TP + FN)$
F1 Score	Harmonic means of precision and recall; balances false positives/negatives.	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$
Confusion Matrix	Table showing counts of TP, FP, FN, TN; used to compute other metrics.	Not a formula; it's a 2×2 (or NxN) matrix

Confusion matrix :-

True label	Pass	TP	FN.
	Fail	FP	TN
		pass	fail.
		predicted label.	

TP: Predicted Pass / Actual pass

TN: Predicted fail / fail

FP: " Pass / fail

FN: " fail / pass.

10.2 Regression Metrics

Metric	What It Is / Use	How It's Calculated
Mean Absolute Error (MAE)	Average of absolute differences between actual and predicted; easy to interpret.	$(1/n) \sum y - \hat{y} $
Mean Squared Error (MSE)	Penalizes larger errors more than MAE; useful when large errors are worse.	$(1/n) \sum (y - \hat{y})^2$
Root Mean Squared Error (RMSE)	Square root of MSE; same unit as output, easier to interpret than MSE.	$\sqrt{\text{MSE}}$
R² Score (R-squared)	Proportion of variance in target explained by the model; shows goodness of fit.	$1 - (\sum (y - \hat{y})^2 / \sum (y - \bar{y})^2)$

10.3 Clustering Metrics

Metric	What It Is / Use	How It's Calculated
Silhouette Score	Measures how similar a point is to its own cluster vs. others; good for cluster quality.	$(b - a) / \max(a, b)$, where a = intra, b = nearest-cluster dist

Inertia (Within Cluster Sum of Squares)	Measures how tightly data points are clustered around centroids (used in K-Means).	Sum of squared distances between points and their cluster center
--	--	--